LLM Judge Bias and Sensitivity

Inference Pipeline — Architecture Document

ETH Zürich — Training Better Preference Models

Swiss AI Stack · Multi-Model Support

March 2026

Abstract

This document describes the architecture of a research pipeline for measuring systematic biases and sensitivities in LLM-as-a-judge evaluations. The system is a pure HTTP client that dispatches judge queries to the Swiss AI Stack, an OpenAI-compatible inference endpoint hosted at CSCS. We detail the data pipeline, the four core experiments (position bias, template sensitivity, reasoning depth, and input sensitivity), and an OPRO-based prompt optimisation module that automatically discovers system prompts with reduced positional bias.

Contents

1 Overall Architecture

The pipeline is a **pure HTTP client** that sends judge queries to the Swiss AI Stack at https://api.swissai.cscs.ch/v1. There is no local GPU cluster, no Slurm scheduler, and no local vLLM instance. All inference is performed remotely via OpenAI-compatible REST requests; the local code handles data loading, prompt construction, asynchronous dispatch, response parsing, metric computation, and prompt optimisation.

Key terminology. Every API call sends exactly one *system prompt* and one *user prompt*. The system prompt is a hidden instruction that defines the judge's persona and output format (e.g. "You are an expert rater…"). The user prompt contains the evaluation data: the original user question and two candidate responses labelled A and B. The system prompt varies across experiments; the user prompt varies across dataset pairs.

1.1 Data-Flow Overview

Figure 1 illustrates the high-level data flow. The dataset module downloads and caches preference data locally. At experiment time, pairs are loaded from disk, combined with prompt templates, and dispatched as asynchronous HTTP requests. Responses are parsed, saved as JSONL, and fed to the metrics module for analysis.

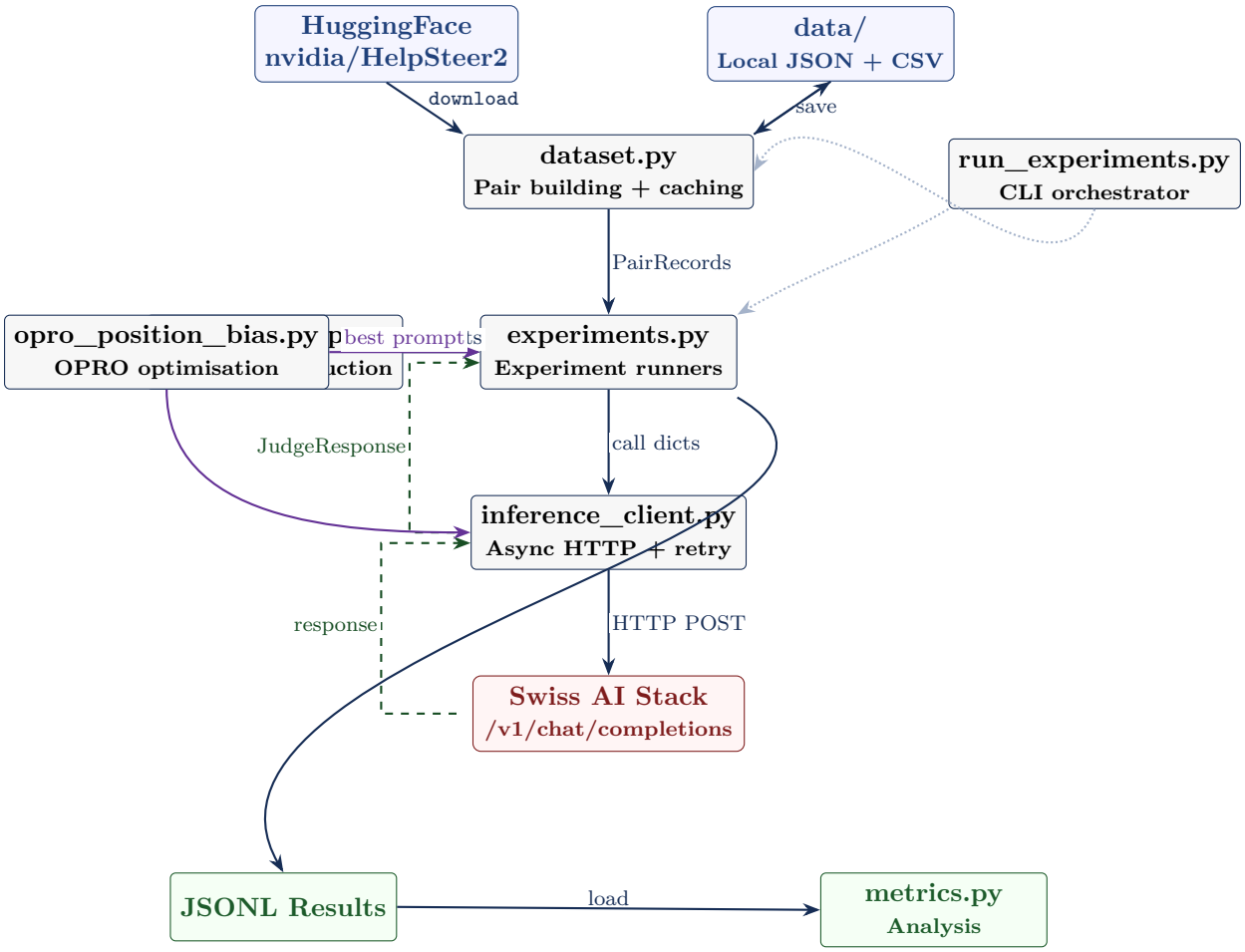

Figure 1: High-level data flow of the evaluation pipeline. Solid arrows indicate the forward path; dashed arrows show the return path. Purple arrows represent the OPRO prompt optimisation loop.

2 Module Descriptions

2.1 `dataset.py` — Data Pipeline

This module handles all interactions with the HelpSteer2 dataset. The pipeline operates in two phases:

Download phase. The `download_dataset()` function fetches HelpSteer2 from HuggingFace, groups responses by prompt, and constructs *all pairwise combinations* within each group. For each pair, a composite quality score is computed as

$$s = \frac{\text{helpfulness} + \text{correctness} + \text{coherence}}{3}.$$

The higher-scoring response is placed as A (gold label = "A"); pairs with equal scores receive gold label "C". Results are saved locally as both JSON and CSV in the `data/` directory.

Two versions are produced per split:

- **Standard**: `prompt_id`, `prompt`, `response_a`, `response_b`, `gold_label`.
- **Full**: additionally includes per-response scores (helpfulness, correctness, coherence, complexity, verbosity for both A and B), composite scores, score gap, response lengths, and a `difficulty` label (easy if gap > 1.0, medium if > 0.33, hard otherwise).

Load phase. At experiment time, `load_dataset_pairs()` reads from local JSON, applies a seeded shuffle, and returns up to n `PairRecord` objects. No network access is required at this stage.

The `PairRecord` dataclass contains five fields: `prompt_id`, `prompt`, `response_a`, `response_b`, and `gold_label`. The `flipped()` method returns a copy with A and B swapped and the gold label inverted (A $\leftrightarrow$ B, C unchanged).

2.2 `inference_client.py` — Async HTTP Client

This module wraps the Swiss AI Stack's OpenAI-compatible endpoint with structured request/response handling, concurrency control, and retry logic.

Configuration. The `InferenceConfig` dataclass holds the endpoint URL, API key, model identifier, generation parameters (`max_tokens`=2048, `temperature`=0.0), and retry settings. The model is configurable; `check_models.py` queries the `/v1/models` endpoint to list currently available models.

Label extraction. After receiving a response, the module extracts the judge verdict through a three-tier cascade:

1. Explicit verdict lines (e.g. "Verdict: B", "Answer: A").
2. A bare A, B, or C at the end of the text.
3. Fallback: the last standalone A/B/C anywhere in the text.

If the model produced a `<think>...</think>` block, it is stripped before extraction and stored separately.

Concurrency and retries. The `SwissAIClient` uses an `asyncio.Semaphore` (default 8) to bound the number of concurrent HTTP requests. On rate-limit responses (HTTP 429) or connection errors, it retries with exponential backoff: delays of 2, 4, 8, 16, 32 seconds, capped at 60 seconds, for up to 5 attempts.

2.3 `templates.py` — Prompt Templates

All system prompts are defined here and registered in a `TEMPLATES` dictionary. Table 1 lists the available templates.

Table 1: System prompt templates used across experiments.

Template ID	Framing	Experiment
`expert_rater`	Baseline expert rater	B1, B2, B3, B4
`llm_judge`	"LLM used to judge"	B2
`neutral`	Imperative, no persona	B2
`academic`	Formal quality evaluator	B2
`minimal`	One-liner	B2
`reason_then_judge`	Explain reasoning, then verdict	B3
`structured_reasoning`	Rate on criteria, then verdict	B3
`expert_rater_alt1-3`	Minor wording variants	B4
`opro`	OPRO-optimised (low position bias)	B1

Five evaluation criteria are supported: `helpful`, `quality`, `accurate`, `harmless`, and `concise`, each mapping to a natural-language phrase. The `build_prompt()` function resolves a template ID and criterion into a (system prompt, user prompt) pair. The user prompt follows a fixed format:

```
[User Prompt]
{prompt}

[Response A]
{response_a}

[Response B]
{response_b}

Which response is {criterion}?
```

2.4 `experiments.py` — Experiment Runners

Four asynchronous functions implement the core experiments. Each builds a list of call dictionaries, dispatches them via `batch_judge()`, and saves results to JSONL. Table 2 summarises the experimental conditions.

Table 2: Experiment functions and their conditions.

Function	ID	Conditions
`run_position_bias`	B1	AB (original), BA (flipped)
`run_template_sensitivity`	B2	5 templates
`run_reasoning_depth`	B3	3 reasoning conditions
`run_input_sensitivity`	B4	$4 \times 5 = 20$ (templates $\times$ criteria)

2.5 `metrics.py` — Metric Computation

This module loads JSONL results and computes experiment-specific metrics:

- `compute_position_bias`: consistency rate, bias direction, and positional preference breakdown.

- `compute_pairwise_agreement`: overall and per-pair agreement across conditions (used for B2 and B4).
- `compute_thinking_accuracy`: accuracy versus gold labels, agreement with the no-reasoning baseline, and average latency per condition (used for B3).

2.6 CLI Orchestrators

`run_experiments.py` is the main entry point. It loads the dataset, configures the inference client, runs selected experiments, computes metrics, and prints summaries. Key flags include `-experiments`, `-n-pairs`, `-template` (including `opro`), `-model`, and `-concurrency`.

`run_opro.py` runs the OPRO optimisation loop (Section 3.5). It loads training and validation pairs, invokes the optimiser, and prints the best prompt with its scores.

`check_models.py` queries the `/v1/models` endpoint and lists all currently available models on the Swiss AI Stack—useful since model availability changes frequently.

3 Experiments

3.1 B1: Position Bias

Research question. Does the LLM judge prefer whichever response appears in a particular position, regardless of content quality?

Method. For each pair, the judge is queried twice: once with the original ordering (condition AB) and once with responses swapped (condition BA). A consistent judge should prefer the same underlying response in both orderings—if the AB verdict is "A", the BA verdict should be "B".

Metrics.

- *Position consistency*: fraction of pairs where the flipped verdict matches the expected inversion.
- *Bias toward first/second position*: fraction where the model always selects the response in position A (or B), regardless of content.
- *Tie inconsistency*: cases where one ordering produces a tie and the other does not.

3.2 B2: Template Sensitivity

Research question. How sensitive is the judge's verdict to the framing of the system prompt, given identical data and response ordering?

Method. Each pair is judged with five system prompt templates (Table 1), keeping data, order, and criterion constant. Only the judge persona changes.

Metrics. Overall pairwise agreement across templates, label distribution per template, and identification of the most volatile pairs (lowest cross-template agreement).

3.3 B3: Reasoning Depth

Research question. Does explicitly prompting the model to reason before giving its verdict improve accuracy against human gold labels?

Method. Three conditions are compared, differing only in the system prompt:

1. *No reasoning*: output only A, B, or C.
2. *Reason then judge*: explain strengths and weaknesses, then give a verdict.
3. *Structured reasoning*: rate each response on helpfulness, accuracy, and coherence, then give a verdict.

All reasoning is prompt-elicited—the prompt itself requests the reasoning. No native API thinking-budget feature is used.

Metrics. Accuracy versus gold labels per condition, delta accuracy over the no-reasoning baseline, and average latency (reasoning conditions produce longer outputs).

3.4 B4: Input Sensitivity

Research question. How sensitive is the judge to minor paraphrasing of the system prompt and changes in the evaluation criterion?

Method. Four semantically equivalent variants of the `expert_rater` template are crossed with five evaluation criteria, yielding $4 \times 5 = 20$ conditions per pair.

Metrics. Pairwise agreement across conditions and criterion drift (how much the label changes when only the criterion changes, with the template held fixed).

3.5 OPRO: Prompt Optimisation for Position Bias

Goal. Automatically discover a system prompt that minimises position bias, using the LLM itself to iteratively generate and refine candidates.

Method. The optimisation follows the OPRO framework (Optimization by PROmpting):

1. Evaluate two seed prompts (`expert_rater` and `llm_judge`) on a fixed subset of training pairs, measuring position consistency.
2. For N iterations: present the LLM with the full history of (prompt $\rightarrow$ score) pairs and ask it to generate a new system prompt. Evaluate the candidate on the same training subset.
3. Select the prompt with the highest training score.
4. Validate on a held-out set to assess generalisation.

A fixed subset of training pairs (default 80) is used for all evaluations, ensuring that scores are comparable across iterations. The best prompt is saved to `results/opro/opro_results.json` and registered as the `opro` template.

Transferability. OPRO-generated prompts are model-specific. A prompt optimised for one model (e.g. Qwen) may not generalise to another (e.g. Llama), as different architectures respond differently to debiasing instructions. When switching models, the optimisation should be re-run.

OPRO usage

```
# Default: 10 iterations, 80 eval pairs, 150 validation pairs
python run_opro.py

# Use the optimised prompt in experiments
python run_experiments.py --template opro --experiments position_bias
```

4 Concurrency and Retry Logic

The pipeline uses Python's `asyncio` event loop with `aiohttp` for non-blocking HTTP. All experiment calls are dispatched simultaneously via `asyncio.gather()`, bounded by an `asyncio.Semaphore`.

Concurrency control. The semaphore (default 8) ensures that at most 8 HTTP requests are in flight at any time, preventing server overload while maintaining high throughput.

Exponential backoff. On HTTP 429 (rate limit) or connection errors, the client waits $\min(\text{base\_delay} \times 2^{\text{attempt}}, 60)$ seconds before retrying. With a base delay of 2 s, the sequence is 2, 4, 8, 16, 32 s (capped at 60 s). After 5 failed attempts, a `RuntimeError` is raised.

Figure 2 illustrates the retry flow.

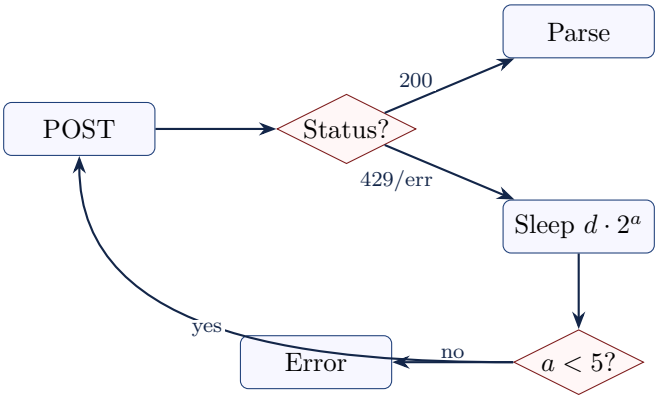

Figure 2: Exponential backoff retry logic. On rate-limit or error responses, the client retries up to 5 times with increasing delays.

5 Output Format

Each experiment writes a JSONL file to the `results/` directory, with one JSON object per judge call. Table 3 describes the schema.

Table 3: Fields in each JSONL result record.

Field	Type	Description
`prompt_id`	str	Unique identifier for the dataset pair
`experiment_id`	str	Experiment name
`condition`	str	Experimental condition (e.g. AB, BA, template name)
`raw_text`	str	Full model output, unmodified
`label`	str | null	Extracted verdict: A, B, C, or null
`thinking`	str | null	Extracted reasoning block, if present
`latency_s`	float	Request latency in seconds
`total_tokens`	int	Total tokens (prompt + completion)
`parse_ok`	bool	Whether label extraction succeeded
`error`	str | null	Error message, if the request failed

The results directory is organised as follows:

```
results/
  position_bias/
    expert_rater_helpful.jsonl
    opro_helpful.jsonl
  template_sensitivity/
    criterion_helpful.jsonl
  reasoning_depth/
    criterion_helpful.jsonl
  input_sensitivity/
    all.jsonl
  opro/
    opro_results.json
```

6 Infrastructure

This pipeline performs no local GPU inference. All model calls are HTTP requests to the Swiss AI Stack. Table 4 summarises the infrastructure.

Table 4: Infrastructure summary.

Component	Detail
Inference backend	Swiss AI Stack (CSCS)
Endpoint	`https://api.swissai.cscs.ch/v1`
API compatibility	OpenAI `/v1/chat/completions`
Model	Configurable (use `check_models.py`)
Authentication	Bearer token via `SWISSAI_API_KEY`
Client library	`aiohttp` (async)
Concurrency	Semaphore-bounded (default 8)
Retry strategy	Exponential backoff, max 5 retries
Local requirements	Python 3.10+, aiohttp, datasets, numpy
Local GPU / cluster	None

7 Dataset Analysis

The following figures are generated by `dataset_analysis.ipynb` and provide an overview of the HelpSteer2 dataset used in all experiments.

7.1 Rating Distributions and Correlations

Figure 3: Distribution of helpfulness, correctness, coherence, complexity, and verbosity ratings across all responses.

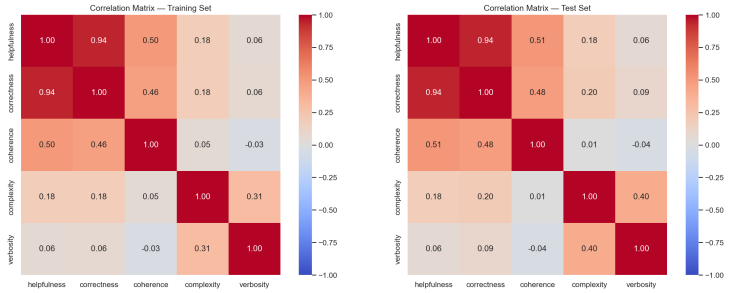

Figure 4: Pairwise Pearson correlation between rating dimensions. Helpfulness and correctness are strongly correlated, suggesting these criteria capture overlapping aspects of response quality.

7.2 Response Length

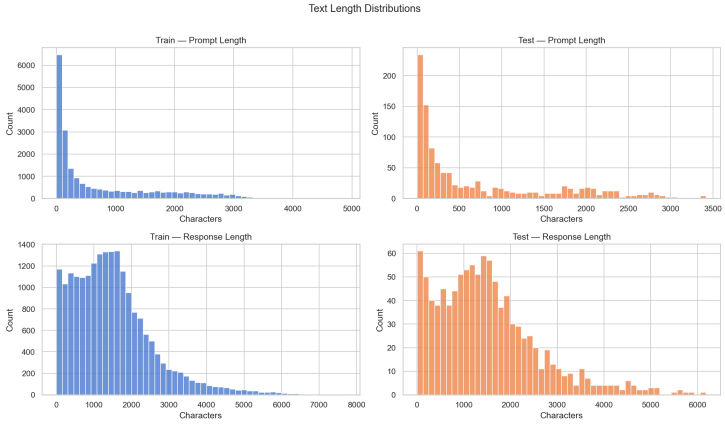

Figure 5: Distribution of prompt and response lengths (in characters).

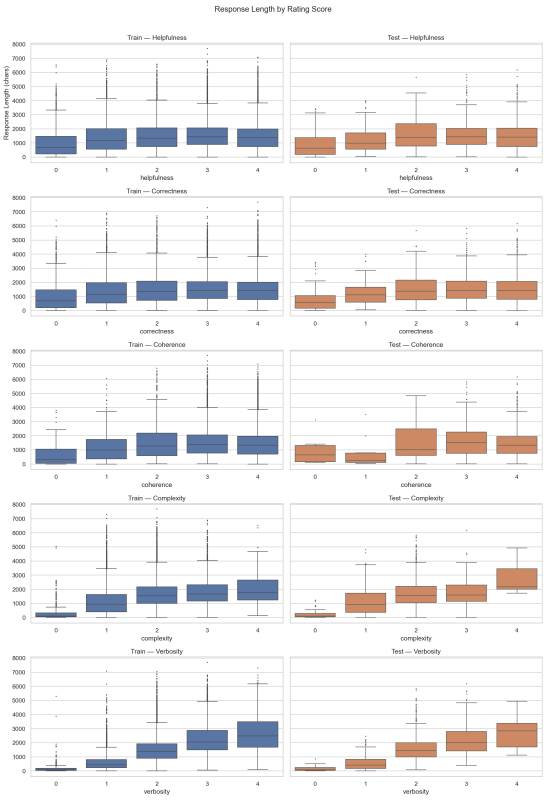

Figure 6: Response length by helpfulness rating. Longer responses tend to receive higher scores.

7.3 Metric Gaps Between Paired Responses

Figure 7: Distribution of per-metric gaps (score of A minus score of B) across all pairs. Larger gaps correspond to easier judgments.

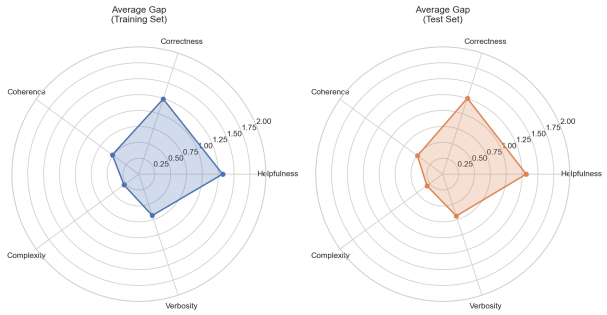

Figure 8: Average metric gaps.

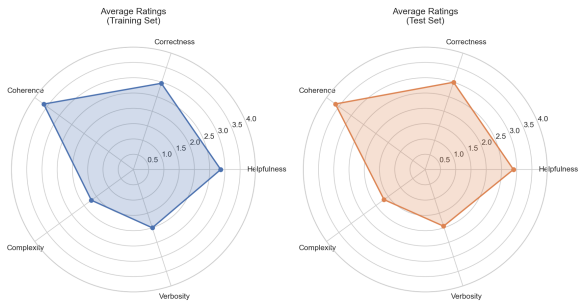

Figure 9: Average ratings.

7.4 Pair-Level Statistics

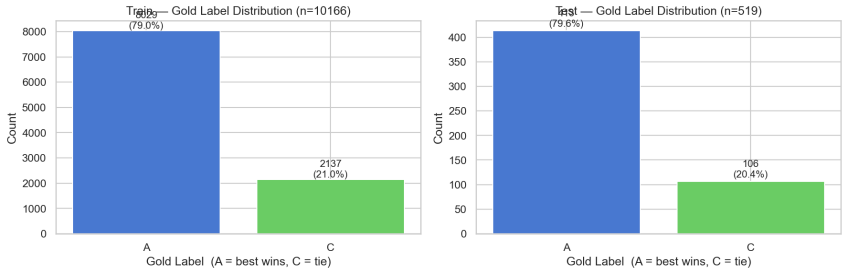

Figure 10: Gold label distribution. The better response is always placed as A, so the majority of labels are A.

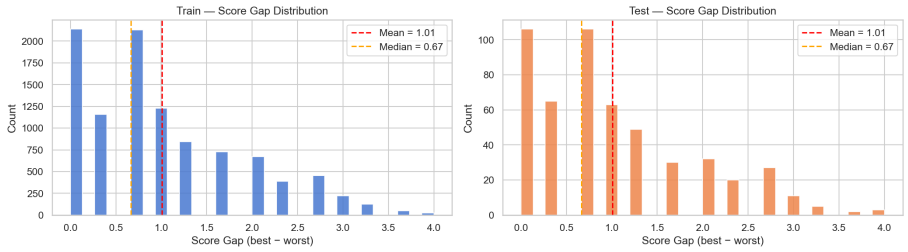

Figure 11: Composite score gap distribution. Smaller gaps indicate pairs that are harder for a judge to distinguish.

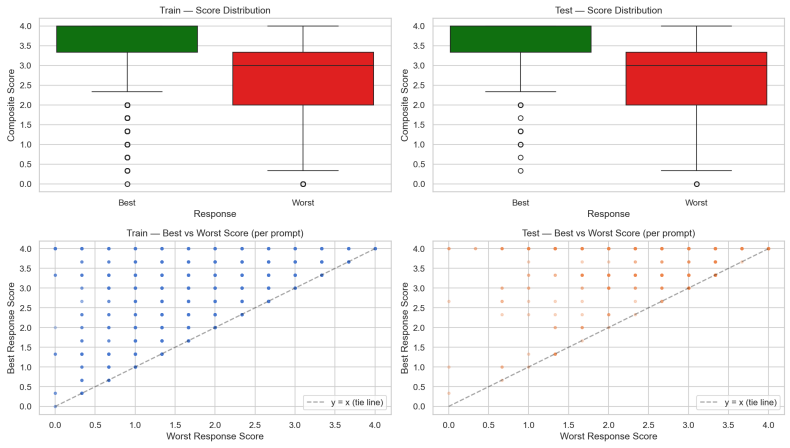

Figure 12: Scatter plot of composite scores: response A versus response B.

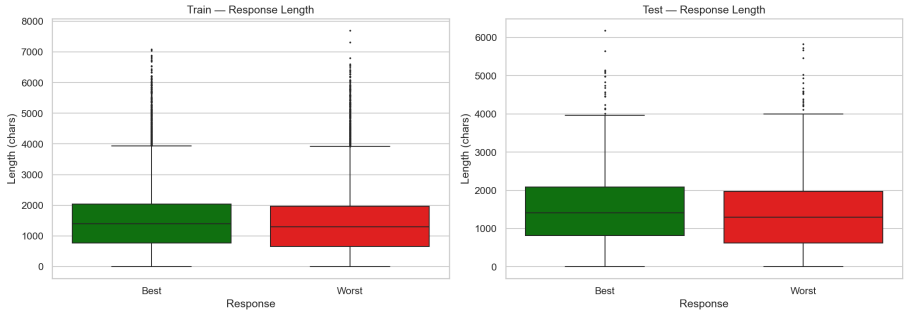

Figure 13: Response length comparison between the better and worse response in each pair.

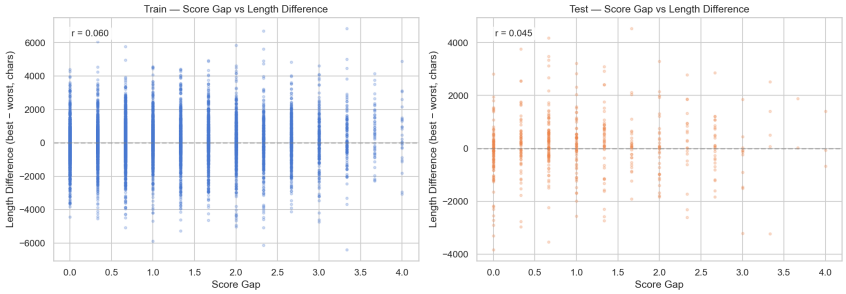

Figure 14: Score gap versus response length difference. Explores whether quality differences correlate with verbosity differences.

7.5 Difficulty buckets

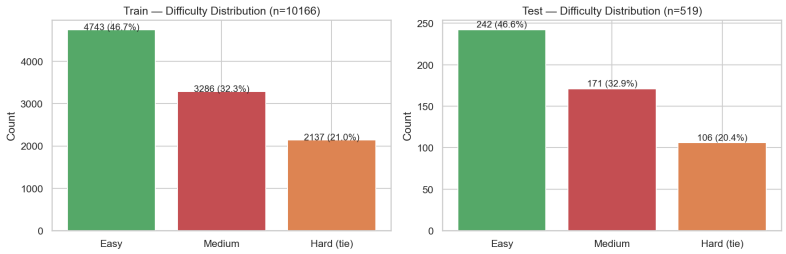

Figure 15: Number of pairs per difficulty bucket. Easy: gap > 1.0; medium: $0.33 <$ gap ≤ 1.0; hard: gap ≤ 0.33.

8 Preliminary Experiments

All experiments use 200 pairs from the HelpSteer2 validation split (seed 42), temperature 0.0, and concurrency 8, with **Qwen3-235B-A22B-Instruct** served via the Swiss AI Stack.

8.1 B1: Position Bias

Table 5 reports position consistency across models and prompt templates. All runs use `run_experiments.py -experiments position_bias -n-pairs 200`. The `opro` prompt was optimised with `run_opro.py -n-train 1000 -eval-pairs 200 -n-val 400 -n-iterations 10` (200 pairs scored per iteration from a pool of 1000 training pairs; 400 held-out pairs for final validation).

Table 5: B1 Position Bias results for Qwen3-235B. Position consistency = fraction of pairs where the flipped verdict matches the expected inversion (higher is better).

Template	Consistency	Bias Rate	Bias → 1st	Bias → 2nd	Tie Incons.
`expert_rater`	0.705	0.295	0.180	0.060	0.055
`opro`	**0.740**	**0.260**	**0.090**	0.130	0.040

Key findings.

- The OPRO-optimised prompt improves position consistency by +3.5 pp over the `expert_rater` baseline (0.740 vs. 0.705).
- The OPRO prompt nearly eliminates first-position bias (0.090 vs. 0.180) but slightly increases second-position bias (0.130 vs. 0.060), suggesting the debiasing instruction overshoots in some cases.
- A small OPRO eval set (80 pairs) caused overfitting; increasing to 200 pairs eliminated the train/validation gap.

8.2 B2: Template Sensitivity

Tables 6 and 7 report template sensitivity results for Qwen3-235B across all five criteria.

Table 6: B2 per-template average pairwise agreement by criterion. Each cell shows how often that template's verdict agrees with the other four templates on average.

Criterion	expert_rater	llm_judge	neutral	academic	minimal
helpful	**0.839**	0.826	0.826	0.792	0.729
accurate	**0.762**	0.735	0.744	0.716	0.527
quality	**0.818**	0.811	**0.818**	0.755	0.714
harmless	**0.748**	0.720	0.734	0.618	0.561
concise	**0.784**	0.755	0.771	0.723	0.660

Key findings.

- `helpful` is the most stable criterion (0.803 overall); `harmless` is the most volatile (0.676), indicating that safety judgements are especially sensitive to prompt framing.
- `expert_rater` is the most consensus-aligned template across all criteria — its verdicts agree most with the other four templates.
- `minimal` is the most deviant template, especially on `accurate` (0.527) and `harmless` (0.561) — likely because the one-liner prompt forces decisive answers that diverge from the more deliberate verdicts of longer prompts.

Table 7: B2 label distribution per template (A / B / C counts out of 200 pairs). Bold C values highlight templates producing the most ties.

Criterion	Template	A	B	C
helpful	expert_rater	146	42	12
	llm_judge	131	47	22
	neutral	138	53	9
	academic	126	45	**29**
	minimal	127	70	3
quality	expert_rater	141	46	13
	llm_judge	119	50	31
	neutral	134	54	12
	academic	111	40	**49**
	minimal	130	70	0
accurate	expert_rater	121	29	50
	llm_judge	103	33	64
	neutral	119	28	53
	academic	90	28	**82**
	minimal	125	73	2
harmless	expert_rater	127	34	39
	llm_judge	94	32	**74**
	neutral	124	39	37
	academic	72	29	**99**
	minimal	122	76	2
concise	expert_rater	107	66	27
	llm_judge	96	59	**45**
	neutral	102	89	9
	academic	91	58	**51**
	minimal	117	82	1

- `academic` consistently produces the most tie labels (C) across all criteria — up to 99 out of 200 pairs for `harmless` — suggesting a formal evaluator persona makes the model more hesitant to commit to a verdict.
- Some pairs achieve a minimum pairwise agreement of 0.2 (4 out of 5 templates disagree), appearing across multiple criteria — indicating a subset of genuinely ambiguous pairs.

8.3 B3: Reasoning Depth

Table 8 reports accuracy versus gold labels for Qwen3-235B across all five evaluation criteria and three reasoning conditions.

Table 8: B3 Reasoning Depth results for Qwen3-235B, 200 validation pairs. Accuracy is fraction of verdicts matching the human gold label.

Criterion	No reasoning	Reason then judge	Structured reasoning
helpful	**0.610**	0.565	0.595
accurate	**0.605**	0.555	0.565
quality	0.615	0.605	**0.615**
harmless	**0.615**	0.600	0.605
concise	0.480	0.500	**0.545**
Average	**0.585**	0.565	**0.585**

Key findings.

- On average, *no reasoning* and *structured reasoning* tie at 0.585, both outperforming *reason then judge* (0.565).
- The pattern is criterion-dependent: for `concise`, structured reasoning helps (+6.5 pp), likely because explicit criteria application benefits conciseness judgements.
- For `quality`, all three conditions are approximately equal.
- For `helpful`, `accurate`, and `harmless`, reasoning consistently hurts slightly.
- Free-form reasoning (*reason then judge*) is strictly worse than direct verdicts across all criteria.
- Reasoning conditions cost 25–50× more latency than no-reasoning on Qwen3-235B, due to longer generated chains.

8.4 Next Experiments

The current experiments use five evaluation criteria (`helpful`, `accurate`, `quality`, `harmless`, `concise`) that were designed independently of the HelpSteer2 annotation schema. However, HelpSteer2 provides per-response human ratings on five specific attributes: *helpfulness*, *correctness*, *coherence*, *complexity*, and *verbosity*.

The current gold labels are derived from a composite score (helpfulness+correctness+coherence)/3, which does not align with any single criterion used in B2, B3, or B4. This creates a mismatch: for example, B3 with `accurate` measures accuracy against a composite gold label rather than purely against human *correctness* scores.

Two approaches are possible. If keeping the current experiment criteria, a natural next step is to re-derive criterion-specific gold labels directly from HelpSteer2 attributes and rerun the experiments:

Experiment criterion	HelpSteer2 attribute
`helpful`	*helpfulness*
`accurate`	*correctness*
`concise`	*verbosity* (inverted)
`quality`	mean of all attributes
`harmless`	no direct mapping

This would make B3 accuracy scores directly interpretable and allow a fairer comparison across criteria — potentially changing current findings on whether and when reasoning helps.

Alternatively, if the goal is to test different criteria altogether, one could evaluate judge verdicts against HelpSteer2 gold labels derived *per-attribute* (e.g. which response has higher *correctness*?) as well as against the *average* composite score. This would reveal whether judge performance and bias patterns vary depending on which human annotation dimension is treated as ground truth.

End of document.